UNIT 2

TCP client/server

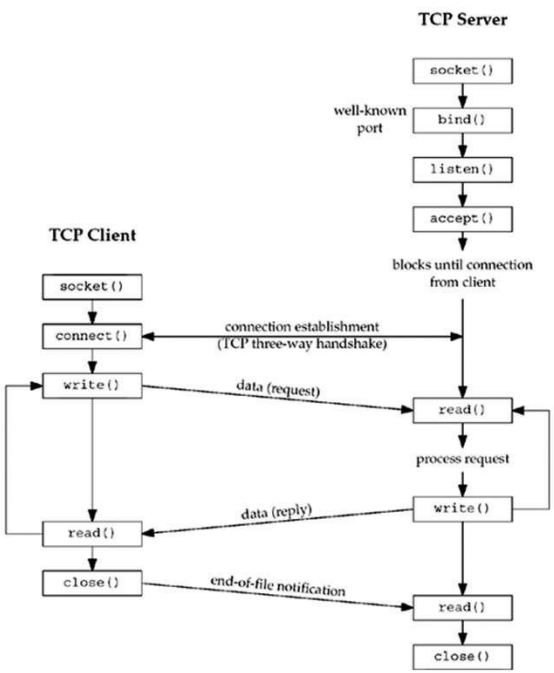
Contents

1. Socket function
2. connect function, bind, listen, accept, fork, exec functions
3. Concurrent Server
4. close function
5. getsockname and getpeername functions
6. TCP Echo server – main – str_echo
7. TCP Echo client - main – str_echo
8. Normal startup, normal termination

Socket Functions

- Socket functions required to write a complete TCP client and server, along with concurrent servers.
- Each client connection causes the server to fork a new process just for that client.
- A timeline of the typical scenario that takes place between a TCP client and server.

First, the server is started, then sometime later, a client is started that connects to the server.



Socket Functions

- The client sends a request to the server, the server processes the request, and the server sends a reply back to the client.
- This continues until the client closes its end of the connection, which sends an end-of-file notification to the server.
- The server then closes its end of the connection and either terminates or waits for a new client connection.

socket()

- To perform network I/O, the first thing a process must do is call the socket function, specifying the type of communication protocol desired (TCP using IPv4, UDP using IPv6, Unix domain stream protocol, etc.).

```
#include <sys/socket.h>

int socket (int family, int type, int protocol);

/* Returns: non-negative descriptor if OK, -1 on error */
```

socket()

Arguments:

family specifies the protocol family and is one of the constants in the table below.

family	Description
AF_INET	IPv4 protocols
AF_INET6	IPv6 protocols
AF_LOCAL	Unix domain protocols (Chapter 15)
AF_ROUTE	Routing sockets (Chapter 18)
AF_KEY	Key socket (Chapter 19)

This argument is often referred to as domain instead of family.

socket()

Arguments:

The socket *type* is one of the constants shown in table below:

type	Description
SOCK_STREAM	stream socket
SOCK_DGRAM	datagram socket
SOCK_SEQPACKET	sequenced packet socket
SOCK_RAW	raw socket

This argument is often referred to as domain instead of family.

socket()

Arguments:

The *protocol* argument to the socket function should be set to the specific protocol type found in the table below, or 0 to select the system's default for the given combination of *family* and *type*.

protocol	Description
IPPROTO_TCP	TCP transport protocol
IPPROTO_UDP	UDP transport protocol
IPPROTO_SCTP	SCTP transport protocol

socket()

Arguments:

Not all combinations of socket *family* and *type* are valid.

	AF_INET	AF_INET6	AF_LOCAL	AF_ROUTE	AF_KEY
SOCK_STREAM	TCP/SCTP	TCP/SCTP	Yes		
SOCK_DGRAM	UDP	UDP	Yes		
SOCK_SEQPACKET	SCTP	SCTP	Yes		
SOCK_RAW	IPv4	IPv6		Yes	Yes

- On success, the socket function returns a small non-negative integer value, similar to a file descriptor. This is called a **socket descriptor**, or a *sockfd*.

connect()

The connect function is used by a TCP client to establish a connection with a TCP server.

```
#include <sys/socket.h>

int connect(int sockfd, const struct sockaddr *servaddr, socklen_t addrlen);

/* Returns: 0 if OK, -1 on error */
```

- *sockfd* is a socket descriptor returned by the socket function.
- The *servaddr* and *addrlen* arguments are a pointer to a socket address structure (which contains the IP address and port number of the server) and its size.

Note: <u>The client does not have to call bind before calling connect: the kernel will choose both an ephemeral port and the source IP address if necessary.</u>

connect()

- In the case of a TCP socket, the connect function initiates TCP's three-way handshake .
- The function returns only when the connection is established or an error occurs. There are several different error returns possible:
  - If the client TCP receives no response to its SYN segment, **ETIMEDOUT** is returned.
  - If the server's response to the client's SYN is a reset (**RST**), this indicates that no process is waiting for connections on the server host at the port specified (the server process is probably not running). This is a **hard error** and the error ECONNREFUSED is returned to the client as soon as the RST is received.
  - If the client's SYN elicits an **ICMP "destination unreachable"** from some intermediate router, this is considered a **soft error**.

    either EHOSTUNREACH or ENETUNREACH

bind()

- The bind function **assigns a local protocol address to a socket**. The protocol address is the combination of either a 32-bit IPv4 address or a 128-bit IPv6 address, along with a 16-bit TCP or UDP port number.

```
#include <sys/socket.h>

int bind (int sockfd, const struct sockaddr *myaddr, socklen_t addrlen);

/* Returns: 0 if OK,-1 on error */
```

- The second argument *myaddr* is a pointer to a protocol-specific address
- The third argument *addrlen* is the size of this address structure.

<u>With TCP, calling bind lets us specify a port number, an IP address, both, or neither.</u>

bind()

- **Servers bind their well-known port when they start.**
- If a TCP client or server does not do this, the kernel chooses an ephemeral port for the socket when either connect or listen is called.
  - It is normal for a TCP client to let the kernel choose an ephemeral port, unless the application requires a reserved port
  - However, it is rare for a TCP server to let the kernel choose an ephemeral port, since servers are known by their well-known port.

bind()

- As mentioned, calling bind lets us specify the IP address, the port, both, or neither.
- The following table summarizes the values to which we set sin_addr and sin_port, or sin6_addr and sin6_port, depending on the desired result.

IP address	Port	Result
Wildcard	0	Kernel chooses IP address and port
Wildcard	nonzero	Kernel chooses IP address, process specifies port
Local IP address	0	Process specifies IP address, kernel chooses port
Local IP address	nonzero	Process specifies IP address and port

- If we specify a port number of 0, the kernel chooses an ephemeral port when bind is called.
- If we specify a wildcard IP address, the kernel does not choose the local IP address until either the socket is connected (TCP) or a datagram is sent on the socket (UDP).

bind()

Wildcard Address and INADDR_ANY *

- With IPv4, the *wildcard* address is specified by the constant INADDR_ANY, whose value is normally 0. This tells the kernel to choose the IP address.

```
struct sockaddr_in   servaddr;
servaddr.sin_addr.s_addr = htonl (INADDR_ANY);     /* wildcard */
```

```
struct sockaddr_in6    serv;
serv.sin6_addr = in6addr_any;     /* wildcard */
```

listen()

- The listen function is called only by a TCP server and it performs two actions:

1. The listen function **converts an unconnected socket into a passive socket**, indicating that the kernel should accept incoming connection requests directed to this socket.
2. The second **argument backlog to this function** specifies the maximum number of connections the kernel should queue for this socket.

❖ This function is normally called after both the socket and bind functions and must be called before calling the accept function.

```
#include <sys/socket.h>

#int listen (int sockfd, int backlog);
```

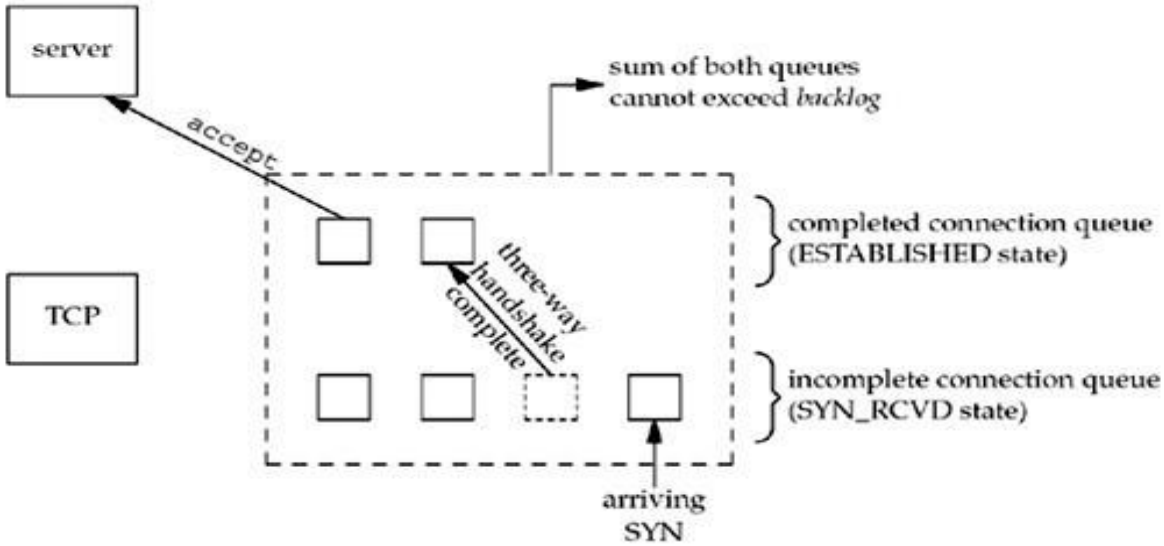
Returns: 0 if OK, -1 on error

listen()

Connection queues *- To understand the backlog argument

- The listening socket, the kernel maintains two queues:

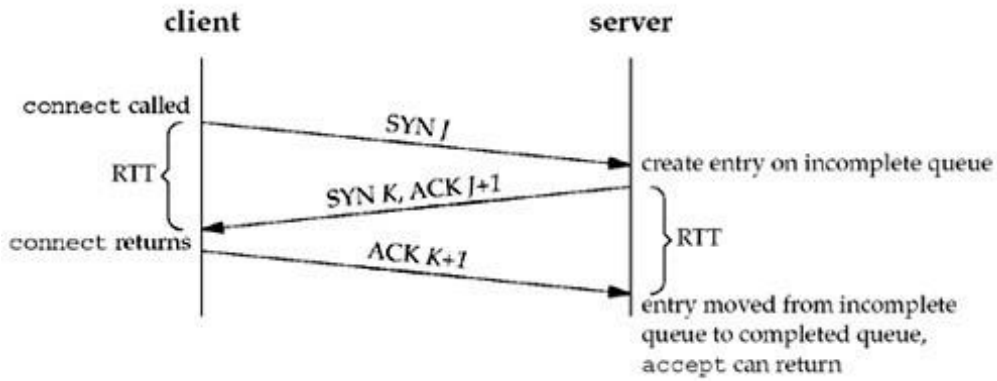
1. ***An incomplete connection queue***, which contains an entry for each SYN that has arrived from a client for which the server is awaiting completion of the TCP three-way handshake. These sockets are in the SYN_RCVD state.
2. A ***completed connection queue***, which contains an entry for each client with whom the TCP three-way handshake has completed. These sockets are in the ESTABLISHED state



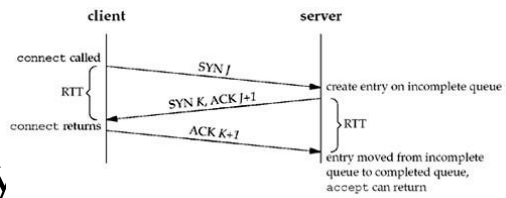
listen()

Packet exchanges during connection establishment

- The following figure depicts the packets exchanged during the connection establishment with these two queues:



listen()



Packet exchanges during connection establishment *

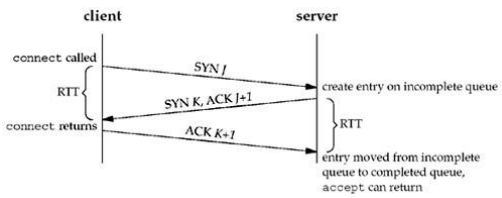
- When a SYN arrives from a client, TCP creates a new entry

the incomplete queue and then responds with the second segment of the three-way handshake: the server's SYN with an ACK of the client's SYN

This entry will remain on the incomplete queue, until:

- The third segment of the three-way handshake arrives (the client's ACK of the server's SYN), or
- The entry times out.
- If the three-way handshake completes normally, the entry moves from the incomplete queue to the end of the completed queue.
- When the process calls accept:
  - The first entry on the completed queue is returned to the process, or
  - If the queue is empty, the process is put to sleep until an entry is placed onto the completed queue.

listen()



The backlog argument *

Several points to consider when handling the two queues:

- Sum of both queues.
- Multiplied by 1.5
- Do not specify value of 0 for backlog, as different implementations interpret this differently If you do not want any clients connecting to your listening socket, close the listening socket.
- An entry remains on the incomplete connection queue for One RTT.
- Configurable maximum value.

listen()

SYN Flooding *

- SYN flooding is a type of attack (the attacker writes a program to send SYNs at a high rate to the victim) that ***attempts to fill the incomplete connection queue*** for one or more TCP ports.
- Additionally, the source IP address of each SYN is set to a random number (called ***IP spoofing***) so that the server's SYN/ACK goes nowhere.
- This also prevents the server from knowing the real IP address of the attacker.
- By filling the incomplete queue with bogus SYNs, legitimate SYNs are not queued, providing a ***denial of service*** to legitimate clients.

accept()

- accept is **called by a TCP server to return the next completed connection** from the front of the completed connection queue.
- If the completed connection queue is empty, the process is put to sleep (assuming the default of a blocking socket).

```
#include <sys/socket.h>

int accept (int sockfd, struct sockaddr *cliaddr, socklen_t *addrlen);

/* Returns: non-negative descriptor if OK, -1 on error */
```

- The *cliaddr* and *addrlen* arguments are used to return the protocol address of the connected peer process (the client).
- *addrlen* is a value-result argument
  - Before the call, we set the integer value referenced by **addrlen* to the size of the socket address structure pointed to by *cliaddr*;
- On return, this integer value contains the actual number of bytes stored by the kernel in the socket address structure.

accept()

- If successful, accept returns a new descriptor automatically created by the kernel.
- This new descriptor refers to the TCP connection with the client.
- The **listening socket** is the first argument (*sockfd*) to accept (the descriptor created by socket and used as the first argument to both bind and listen).
- The **connected socket** is the return value from accept the connected socket.

It is important to differentiate between these two sockets:

- A given server normally creates only <u>one listening socket</u>, which then exists for the lifetime of the server.
- The kernel creates <u>one connected socket</u> for each client connection that is accepted (for which the TCP three-way handshake completes).
- When the server is finished serving a given client, the connected socket is closed.

accept()

This function returns up to three values:

- An integer return code that is either a new socket descriptor or an error indication,
- The protocol address of the client process (through the *cliaddr* pointer),
- The size of this address (through the *addrlen* pointer).

Close()

- Used to close a socket and terminate a TCP connection.

```
#include <unistd.h>

int close (int sockfd);

/* Returns: 0 if OK, -1 on error */
```

- The default action is mark the socket as closed and return to the process immediately.

fork and exec Functions

- fork() and exec() are used in creation of concurrent servers.
- The fork function returns twice.

```
#include <unistd.h>
```

```
pid_t fork(void);
```

Returns: 0 in child, process ID of child in parent, -1 on error

- It returns once in the calling process (called the parent) with a return value that is the process ID of the newly created process (the child).
- It also returns once in the child, with a return value of 0.
- The return value tells the process whether it is the parent or the child.
- All descriptors open in the parent before the call to fork are shared with the child after fork returns.

fork and exec Functions

- In network servers, the parent **calls accept and then calls fork**.
- The connected socket is then shared between the parent and child.
- Normally, the **child then reads and writes** the connected socket and the **parent closes the connected socket**.
- There are two typical uses of fork:
  1. A process **makes a copy of itself** so that one copy can handle one operation while the other copy does another task.
  2. A process **wants to execute another program**. Since the only way to create a new process is by calling fork, the process first calls fork to make a copy of itself, and then one of the copies (typically the child process) **calls exec** (described next) to replace itself with the new program.

fork and exec Functions

- The only way in which an executable program file on disk can be executed by Unix is for an existing process to call one of the six exec functions.
- **exec replaces** the current process image with the new program file, and this new program normally starts at the main function.
- The process ID does not change.
- The process that calls exec as the ***calling process*** and the newly executed program as the ***new program.***

fork and exec Functions

- The differences in the six exec functions are:

  (a) whether the program file to execute is specified by a filename or a pathname

  (b) whether the arguments to the new program are listed one by one or referenced through an array of pointers.

  (c) whether the environment of the calling process is passed to the new program or whether a new environment is specified.

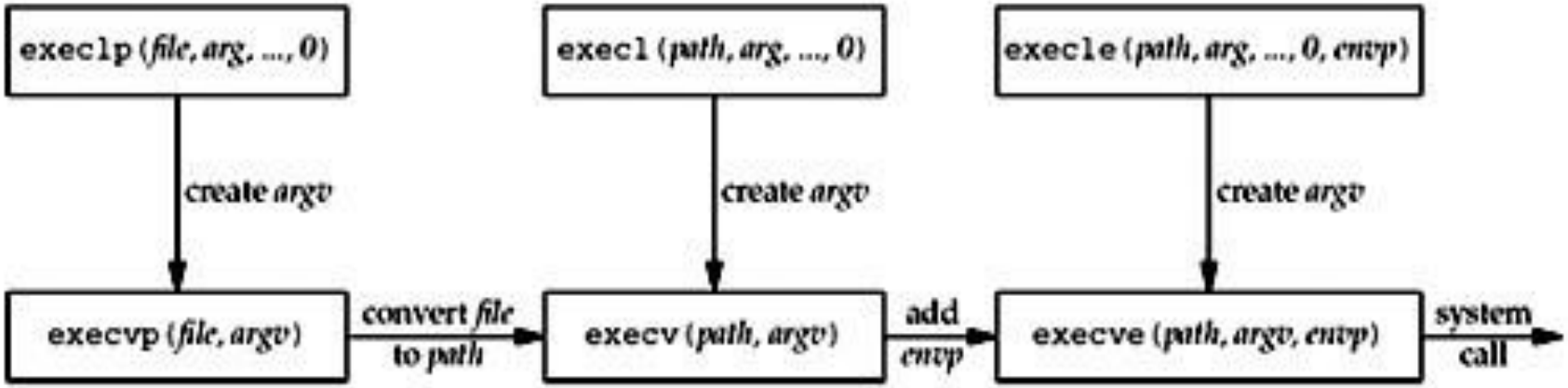
exec Functions

```
#include <unistd.h>

int execl (const char *pathname, const char *arg0, ... /* (char *) 0 */ );

int execv (const char *pathname, char *const argv[]);

int execle (const char *pathname, const char *arg0, ...
                    /* (char *) 0, char *const envp[] */ );

int execve (const char *pathname, char *const argv[], char *const envp[]);

int execlp (const char *filename, const char *arg0, ... /* (char *) 0 */ );

int execvp (const char *filename, char *const argv[]);
```

All six return: -1 on error, no return on success

Descriptors open in the process before calling exec normally remain open across the exec.

exec Functions



Only execve is a system call within the kernel and the other five are library functions that call execve.

exec Functions

Differences among these six functions:

•The **three functions in the top row** specify each argument string as a separate argument to the exec function, with a null pointer terminating the variable number of arguments. The **three functions in the second row** have an argv array, containing pointers to the argument strings. This argv array must contain a null pointer to specify its end, since a count is not specified.

•

•The two functions in **the left column specify a filename argument**. This is converted into a pathname using the current PATH environment variable. If the filename argument to execlp or execvp contains a slash (/) anywhere in the string, the PATH variable is not used. The four functions in the right two columns specify a **fully qualified pathname argument**.

•The four functions in the left two columns **do not specify an explicit environment pointer**. Instead, the current value of the external variable environ is used for building an environment list that is passed to the new program. The two functions in the right column **specify an explicit environment list**. The envp array of pointers must be terminated by a null pointer.

RV College of Engineering®

Go, change the world

Concurrent Servers

Concurrent Server

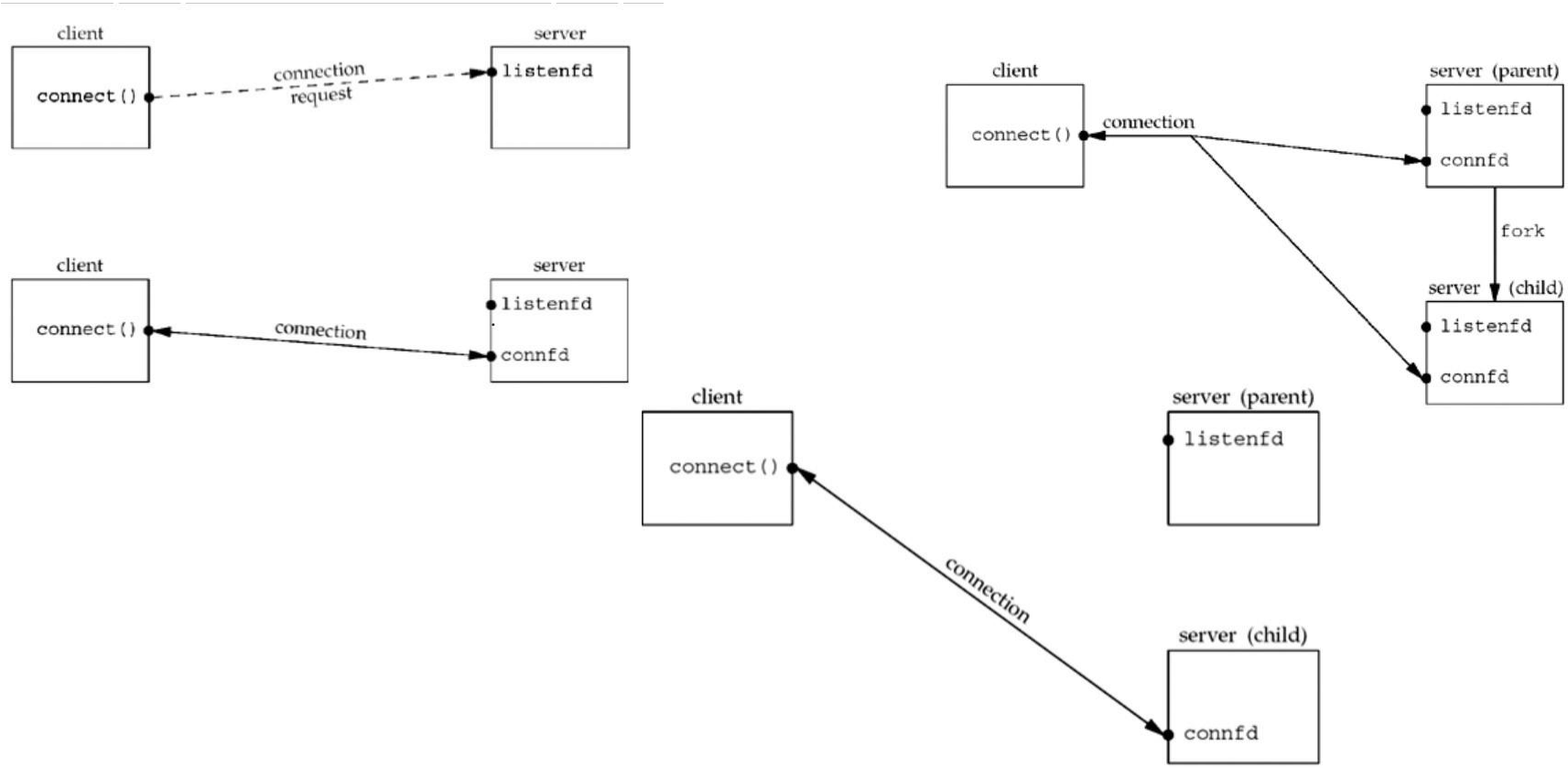
Types of Servers:

- A server can be iterative- it iterates through each client and serves one request at a time.
- A server can be concurrent – it can handle multiple clients at the same time in parallel.
- Necessity??
- when a client request can take longer to service, a server should'nt be tied with one client; it can be designed to handle multiple clients at the same time.
- How??

  –fork a child process to handle each client.

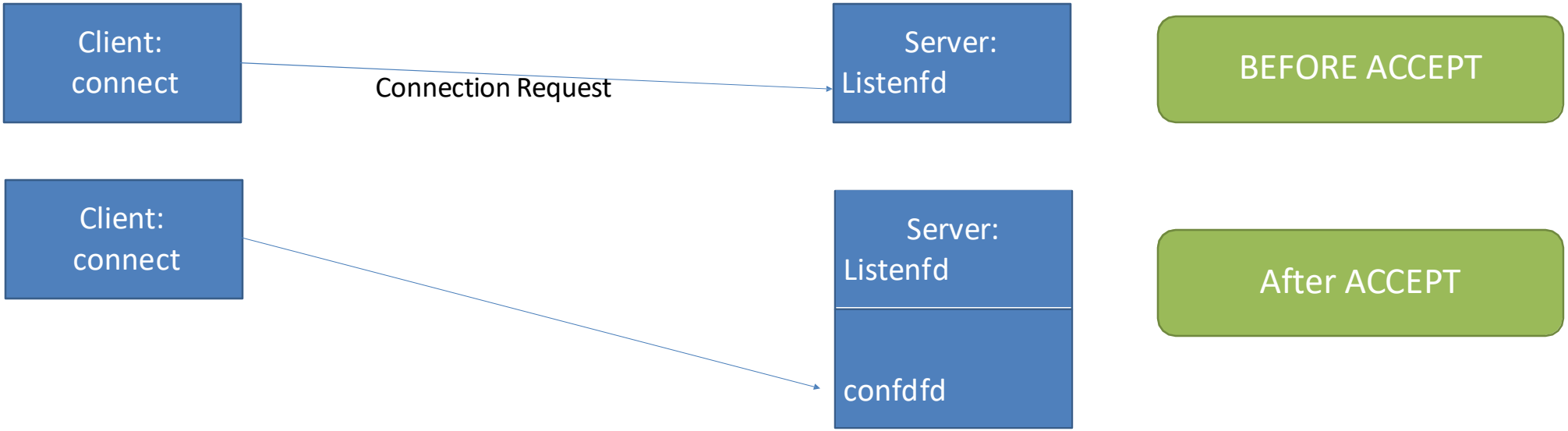
Visualizing the sockets and connection

Before call to accept, the server is blocked to accept the connection

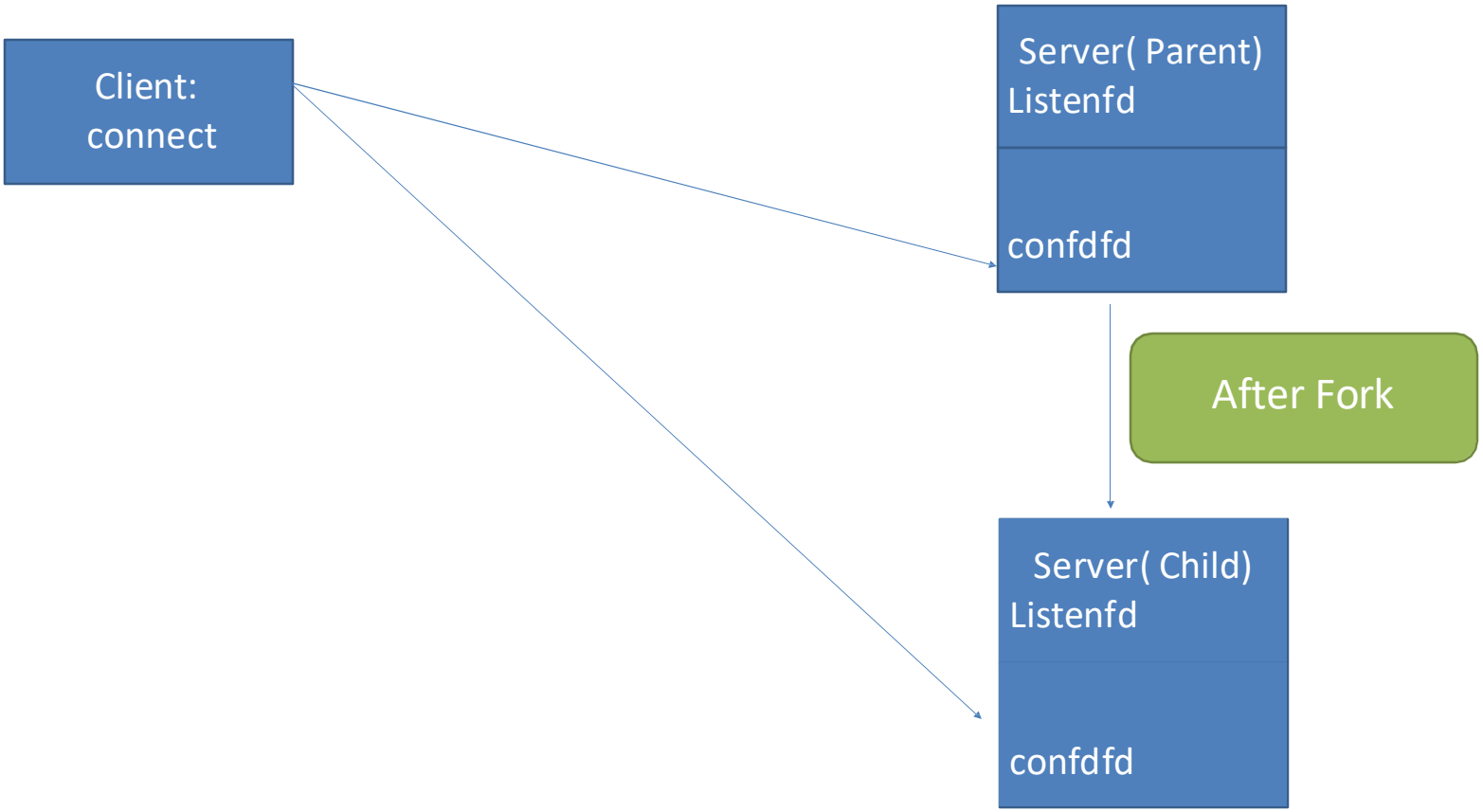


Reference Count

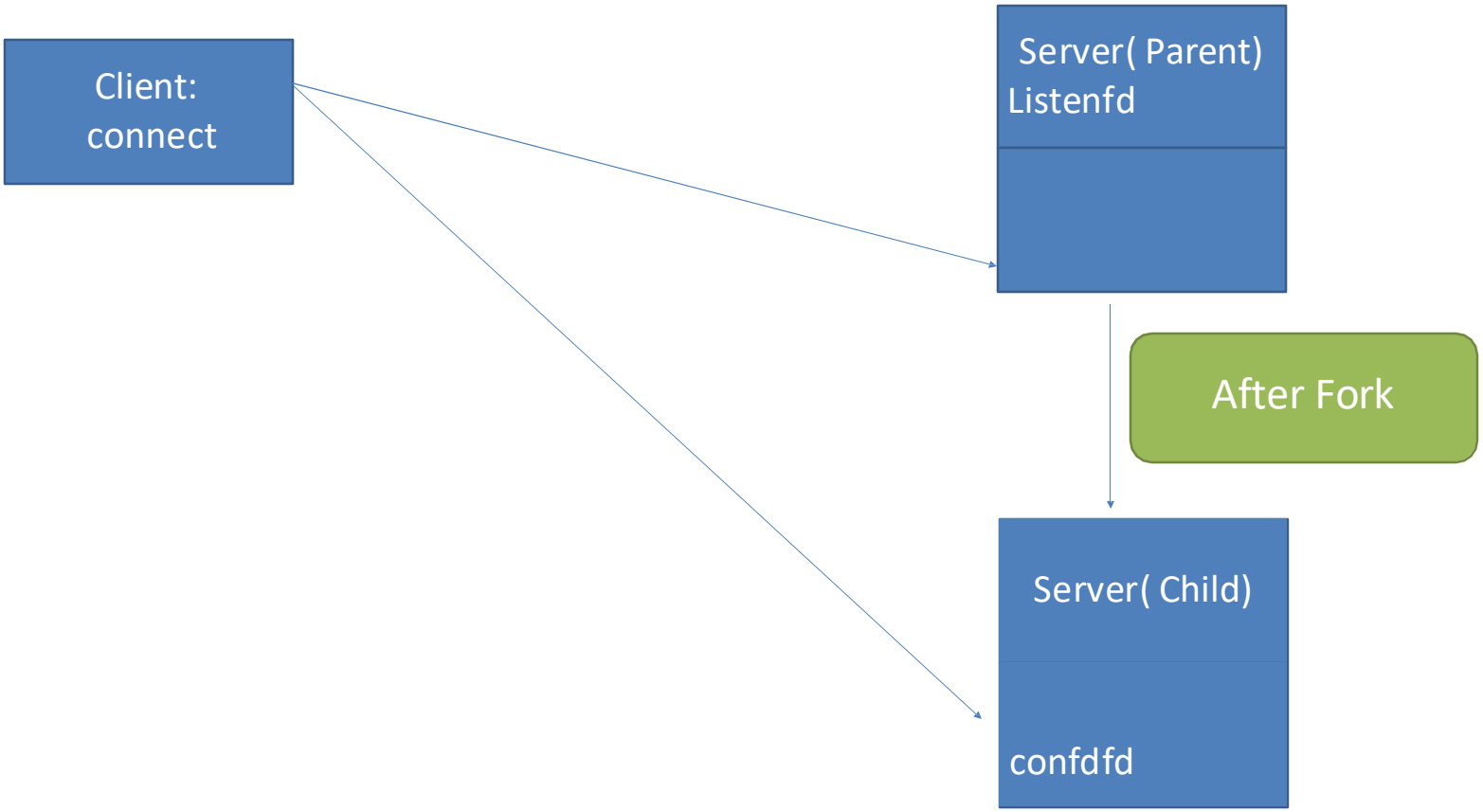
The actual cleanup and de-allocation of the socket does not happen until the reference count reaches 0.



Concurrent Server



Concurrent Server



Outline/Template of a concurrent server

```
pid_t pid;
int listenfd, connfd;
listenfd = Socket( ... );
 /* fill in sockaddr_in{} with server's well-known port */
Bind(listenfd, ... );
Listen(listenfd, LISTENQ);
for ( ; ; ) {
 connfd = Accept (listenfd, ... ); /* probably blocks */
 if( (pid = Fork()) == 0) {
 Close(listenfd); /* child closes listening socket */
 doit(connfd); /* process the request */
 Close(connfd); /* done with this client */
 exit(0); /* child terminates */
 }
 Close(connfd); /* parent closes connected socket */
}
```

Concurrent Servers

- When a connection is established, accept returns, the server calls fork, and the child process services the client (on connfd, the connected socket) and the parent process waits for another connection (on listenfd, the listening socket).

- The parent closes the connected socket since the child handles the new client.

- We assume that the function doit does whatever is required to service the client.

- When this function returns, we explicitly close the connected socket in the child.

- This is not required since the next statement calls exit, and part of process termination is to close all open descriptors by the kernel.

- Whether to include this explicit call to close or not is a matter of personal programming taste.

Reference count of sockets

Calling close on a TCP socket causes a FIN to be sent, followed by the normal TCP connection termination sequence.

Why doesn't the close of connfd by the parent terminate its connection with the client? To understand what's happening, we must understand that every file or socket has a **reference count**.

The reference count is maintained in **the file table entry**. This is a count of the number of descriptors that are currently open that refer to this file or socket.

Reference count of sockets

In the above code:

- After socket returns, the file table entry associated with listenfd has a reference count of 1.
- After accept returns, the file table entry associated with connfd has a reference count of 1.
- But, after fork returns, both descriptors are shared (duplicated) between the parent and child, so the file table entries associated with both sockets now have a reference count of 2.
- Therefore, when the parent closes connfd, it just decrements the reference count from 2 to 1 and that is all.
- The actual cleanup and de-allocation of the socket does not happen until the reference count reaches 0.
- This will occur at some time later when the child closes connfd.

getsockname and getpeername Functions

- getsockname returns the local protocol address associated with a socket.
- getpeername returns the foreign protocol address associated with a socket.

```
#include <sys/socket.h>
int getsockname(int sockfd, struct sockaddr *localaddr, socklen_t *addrlen);
int getpeername(int sockfd, struct sockaddr *peeraddr, socklen_t *addrlen);

/* Both return: 0 if OK, -1 on error */
```

getsockname and getpeername Functions

These two functions are required for the following reasons:

❖ After **connect successfully returns in a TCP client** that does not call bind, *getsockname* returns the local IP address and local port number assigned to the connection by the kernel.

❖ **After calling bind with a port number of 0** (telling the kernel to choose the local portnumber), *getsockname* returns the local port number that was assigned. *getsockname* can be called to obtain the address family of a socket.

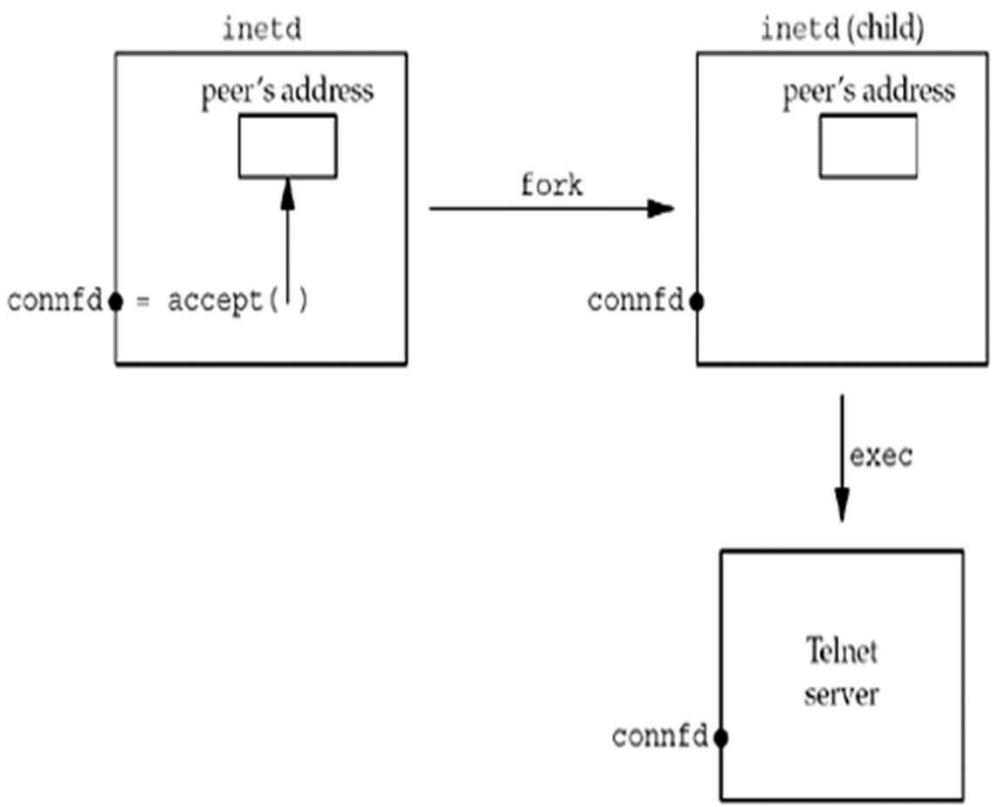
❖ In a **TCP server that binds the wildcard IP address**, once a connection is established with a client (accept returns successfully), the server can call *getsockname* to obtain the local IP address assigned to the connection.

❖ The socket descriptor argument to *getsockname* must be that of the connected socket, and not the listening socket.

❖ When a **server is execed by the process that calls accept**, the only way the server can obtain the identity of the client is to call ***getpeername***.

getsockname and getpeername Functions

For example, *inetd fork*s and *exec*s a TCP server (following figure):



- *inetd* calls *accept*, which returns two values: the connected socket descriptor (connfd, return value of the function) and the "peer's address" (an Internet socket address structure) that contains the IP address and port number of the client.
- *fork* is called and a child of *inetd* is created, with a copy of the parent's memory image, so the socket address structure is available to the child, as is the connected socket descriptor (since the descriptors are shared between the parent and child).
- When the child *exec*s the real server (e.g. Telnet server that we show), the memory image of the child is replaced with the new program file for the Telnet server (the socket address structure containing the peer's address is lost), and the connected socket descriptor remains open across the exec.
- One of the first function calls performed by the Telnet server is *getpeername* to obtain the IP address and port number of the client.

getsockname and getpeername Functions

In this example, the Telnet server must know the value of connfd when it starts. There are two common ways to do this.

The process calling exec pass it as a command-line argument to the newly execed program.

A convention can be established that a certain descriptor is always set to the connected socket before calling exec.

The second one is what inetd does, always setting descriptors 0, 1, and 2 to be the connected socket.

Example: Obtaining the Address Family of a Socket

The sockfd_to_family function shown in the code below returns the address family of a socket.

https://github.com/shichao-an/unpv13e/blob/master/lib/sockfd_to_family.c

```
 int
sockfd_to_family(int sockfd)
{
        struct sockaddr_storage ss;
    socklen_t len;

    len = sizeof(ss);
    if (getsockname(sockfd, (SA *) &ss, &len) < 0)
        return(-1);
    return(ss.ss_family);
}
```

Example: Obtaining the Address Family of a Socket

The sockfd_to_family function shown in the code below returns the address family of a socket.

https://github.com/shichao-an/unpv13e/blob/master/lib/sockfd_to_family.c

This program does the following:

Allocate room for largest socket address structure. Since we do not know what type of socket address structure to allocate, we use a sockaddr_storage value, since it can hold any socket address structure supported by the system.

Call getsockname. We call getsockname and return the address family. The POSIX specification allows a call to getsockname on an unbound socket.

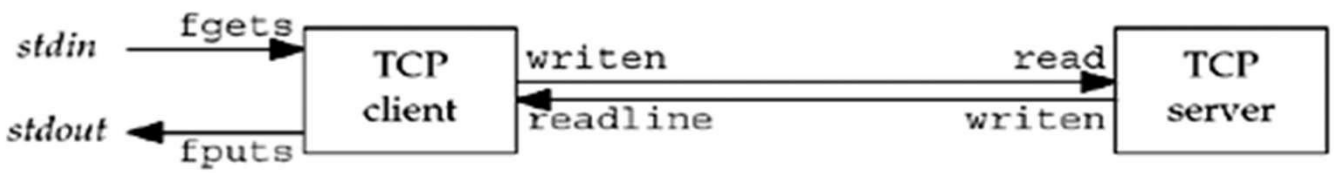
TCP Client/Server

Introduction

We will now use the elementary functions from the previous chapter to write a complete TCP client/server example. Our simple example is an echo server that performs the following steps:

1. The client reads a line of text from its standard input and writes the line to the server.
2. The server reads the line from its network input and echoes the line back to the client.
3. The client reads the echoed line and prints it on its standard output.

The figure below depicts this simple client/server:



TCP Client/Server

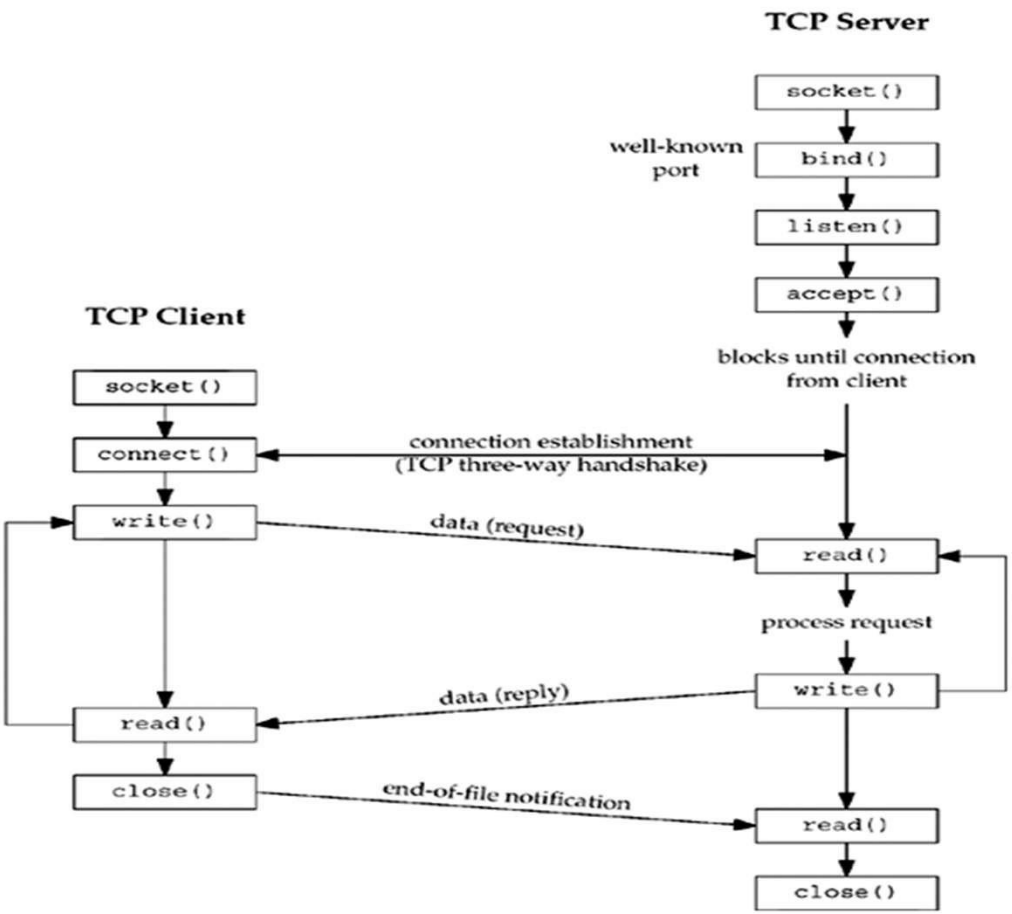
The echo client/server is a valid, simple example of a network application. To expand this example into your own application, all you need to do is change what the server does with the input it receives from its clients.

Besides running the client/server in normal mode (type in a line and watch it echo), we examine lots of boundary conditions:

- What happens when the client and server are started?
- What happens when the client terminates normally?
- What happens to the client if the server process terminates before the client is done?
- What happens to the client if the server host crashes?

TCP Echo Server: main Function

Our TCP client and server follow the flow of functions that we diagrammed



TCP Echo Server: main Function


```
#include "unp.h"
int
main(int argc, char **argv)
{
int listenfd, connfd;
pid_t childpid;
socklen_t clilen;
struct sockaddr_in cliaddr, servaddr;
listenfd = Socket(AF_INET, SOCK_STREAM, 0);
bzero(&servaddr, sizeof(servaddr));
servaddr.sin_family = AF_INET;
servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
servaddr.sin_port = htons(SERV_PORT);
Bind(listenfd, (SA *) &servaddr, sizeof(servaddr));
Listen(listenfd, LISTENQ);
for ( ; ; ) {
clilen = sizeof(cliaddr);
connfd = Accept(listenfd, (SA *) &cliaddr, &clilen);
if ( (childpid = Fork()) == 0) { /* child process */
Close(listenfd); /* close listening socket */
str_echo(connfd); /* process the request */
exit(0);
}
Close(connfd); /* parent closes connected socket */
}
}
```

The below code is the concurrent server program:

The above code does the following:

Create socket, bind server's well-known port

- A TCP socket is created.
- An Internet socket address structure is filled in with the wildcard address (INADDR_ANY) and the server's well-known port (SERV_PORT, which is defined as 9877 in our unp.h header).
- Binding the wildcard address tells the system that we will accept a connection destined for any local interface, in case the system is multihomed.
- Our choice of the TCP port number is based on figure shown in the next slide. *It should be greater than 1023 (we do not need a reserved port), greater than 5000 (to avoid conflict with the ephemeral ports allocated by many Berkeley-derived implementations), less than 49152 (to avoid conflict with the "correct" range of ephemeral ports), and it should not conflict with any registered port.*
- The socket is converted into a listening socket by listen.

TCP Echo Server: main Function

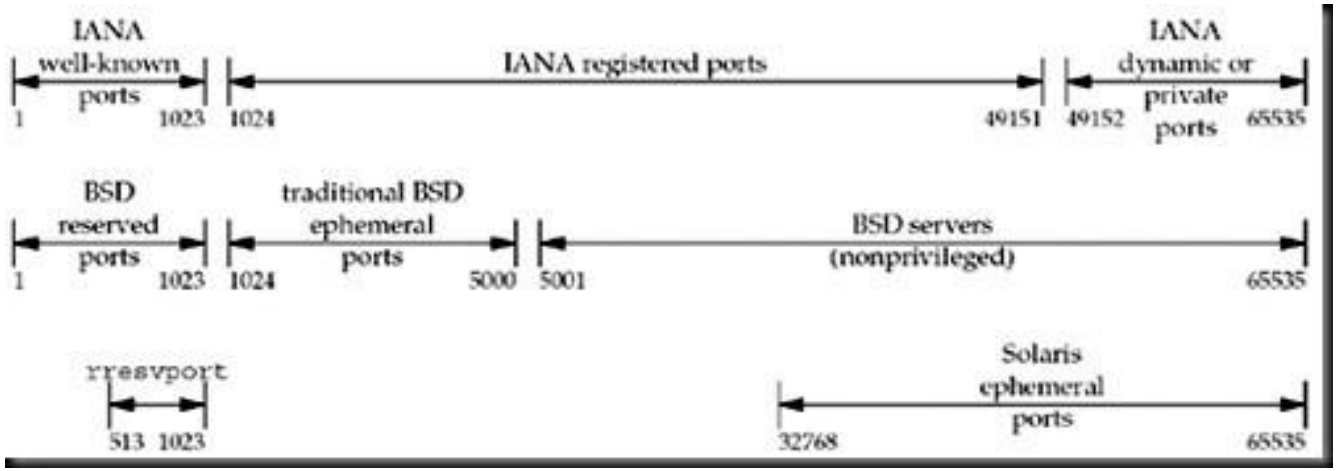


Figure 2.10

Wait for client connection to complete

The server blocks in the call to accept, waiting for a client connection to complete.

Concurrent server

For each client, fork spawns a child, and the child handles the new client. The child closes the listening socket and the parent closes the connected socket. (Section 4.8)

TCP Echo Server: str_echo Function


```
#include "unp.h"
void
str_echo(int sockfd)
{
ssize_t n;
char buf[MAXLINE];
again:
while ( (n = read(sockfd, buf,
MAXLINE)) > 0)
Writen(sockfd, buf, n);
if (n < 0 && errno == EINTR)
goto again;
else if (n < 0)
err_sys("str_echo: read error");
}
```

The function str_echo performs the server processing for each client: It reads data from the client and echoes it back to the client.

The above code does the following:

Read a buffer and echo the buffer:

- read reads data from the socket and the line is echoed back to the client by writen.
- If the client closes the connection (the normal scenario), the receipt of the client's FIN causes the child's read to return 0. This causes the str_echo function to return, which terminates the child.

TCP Echo Client : main Function


```
#include "unp.h"
int
main(int argc, char **argv)
{
int sockfd;
struct sockaddr_in servaddr;
if (argc != 2)
err_quit("usage: tcpcli <IPaddress>");
sockfd = Socket(AF_INET, SOCK_STREAM,
0);
bzero(&servaddr, sizeof(servaddr));
servaddr.sin_family = AF_INET;
servaddr.sin_port = htons(SERV_PORT);
Inet_pton(AF_INET, argv[1],
&servaddr.sin_addr);
Connect(sockfd, (SA *) &servaddr,
sizeof(servaddr));
str_cli(stdin, sockfd); /* do it all */
exit(0);
}
```

The above code does the following:

Create socket, fill in Internet socket address structure

A TCP socket is created and an Internet socket address structure is filled in with the server's IP address and port number. The server's IP address is taken from the command-line argument and the server's well-known port (SERV_PORT) is from our unp.h header.

Connect to server

connect establishes the connection with the server. The function str_cli handles the rest of the client processing.

TCP Echo Client : str_cli Function


```
#include "unp.h"
void
str_cli(FILE *fp, int sockfd)
{
char sendline[MAXLINE], recvline[MAXLINE];

while (Fgets(sendline, MAXLINE, fp) != NULL)
 {
        Writen(sockfd, sendline, strlen(sendline));

        if (Readline(sockfd, recvline, MAXLINE) == 0)
        err_quit("str_cli: server terminated prematurely");

     Fputs(recvline, stdout);
}
}
```

The str_cli function handles the client processing loop: It reads a line of text from standard input, writes it to the server, reads back the server's echo of the line, and outputs the echoed line to standard output.

The above code does the following:

Read a line, write to server

fgets reads a line of text and writen sends the line to the server.

Read echoed line from server, write to standard output

readline reads the line echoed back from the server and fputs writes it to standard output.

Return to main

The loop terminates when fgets returns a null pointer, which occurs when it encounters either an end-of-file (EOF) or an error. Our Fgets wrapper function checks for an error and aborts if one occurs, so Fgets returns a null pointer only when an end-of-file is encountered.

Normal Startup

How the client and server start and end?

What happens when something goes wrong:

- the client host crashes,
- the client process crashes,
- network connectivity is lost

Only by understanding these boundary conditions, and their interaction with the TCP/IP protocols, can we write robust clients and servers that can handle these conditions.

Start the server in the background

First, we start the server in the background:

When the server starts, it calls socket, bind, listen, and accept, blocking in the call to accept.

Normal Startup

Run netstat

Before starting the client, we run the netstat program to verify the state of the server's listening socket.

linux % netstat -a

Active Internet connections (servers and established)

Proto	*Recv-Q*	*Send-Q*	*Local Address*	*Foreign Address*	*State*
tcp	*0*	*0*	**:9877*	**:**	*LISTEN*

- This command shows the status of all sockets on the system. We must specify the **-a flag** to see listening sockets.
- In the output, a socket is in the LISTEN state with a wildcard for the local IP address and a local port of 9877. netstat prints an asterisk for an IP address of 0 (INADDR_ANY, the wildcard) or for a port of 0.

Normal Startup

Start the client on the same host

We then start the client on the same host, specifying the server's IP address of 127.0.0.1 (the loopback address). We could have also specified the server's normal (nonloopback) IP address.

linux % tcpcli01 127.0.0.1

The client calls socket, and connect which causes TCP's three-way handshake. When the three-way handshake completes, connect returns in the client and accept returns in the server. The connection is established. The following steps then take place:

1. The client calls str_cli, which will block in the call to fgets.
2. When accept returns in the server, it calls fork and the child calls str_echo. This function calls readline, which calls read, which blocks while waiting for a line to be sent from the client.
3. The server parent, on the other hand, calls accept again, and blocks while waiting for the next client connection.

Notes from the previous three steps:

- All three processes are asleep (blocked): client, server parent, and server child.
- We purposely list the client step first, and then the server steps when the three-way handshake completes. This is because accept returns one-half of the RTT after connect returns.
  - On the client side, connect returns when the second segment of the handshake is received
  - On the server side, accept does not return until the third segment of the handshake is received.

Normal Startup

Run netstat after connection completes

Since we are running the client and server on the same host, netstat now shows two additional lines of output, corresponding to the TCP connection:

```
linux % netstat -a
Active Internet connections (servers and established)
Proto   Recv-Q   Send-Q   Local Address   Foreign Address   State
tcp     0        0        local host:9877   localhost:42758   ESTABLISHED
tcp     0        0        local host:42758   localhost:9877   ESTABLISHED
Tcp     0        0        *:9877   *:*   LISTEN
```

- The first ESTABLISHED line corresponds to the server child's socket, since the local port is 9877.
- The second ESTABLISHED lines is the client's socket, since the local port is 42758.

If we were running the client and server on different hosts, the client host would display only the client's socket, and the server host would display only the two server sockets.

Normal Startup

Run ps to check process status and relationship

linux % ps -t pts/0 -o pid,ppid,tty,stat,args,wchan

PID	PPID	TT	STAT	COMMAND	WCHAN
22038	22036	pts/6	S	-bash	wait4
17870	22038	pts/6	S	./tcpserv01	wait_for_connect
19315	17870	pts/6	S	./tcpserv01	tcp_data_wait
19314	22038	pts/6	S	./tcpcli01 127.0	read_chan

Very specific arguments to ps are used:

- The TT column (pts/6): client and server are run from the same window, pseudo-terminal number 6.
- The PID and PPID columns show the parent and child relationships.
  - The first tcpserv01 line is the parent and the second tcpserv01 line is the child since the PPID of the child is the parent's PID.
  - The PPID of the parent is the shell (bash).
- The STAT column for all three of our network processes is "S", meaning the process is sleeping (waiting for something).
- The WCHAN column specifies the condition when a process is asleep.
  - Linux prints wait_for_connect when a process is blocked in either accept or connect, tcp_data_wait when a process is blocked on socket input or output, or read_chan when a process is blocked on terminal I/O.
  - In ps(1), WCHAN column indicates the name of the kernel function in which the process is sleeping, a "-" if the process is running, or a "*" if the process is multi-threaded and ps is not displaying threads.

Normal Termination

At this point, the connection is established and whatever we type to the client is echoed back.

```
linux % tcpcli01 127.0.0.1        # we showed this line earlier
hello, world                      # we now type this
hello, world                      # and the line is echoed
good bye
good bye
^D                                # Control-D is our terminal EOF character
```

Normal Termination

If we immediately execute netstat, we have:

```
linux % netstat -a | grep 9877
 tcp        0      0      *:9877                  *:*                    LISTEN
tcp         0      0     localhost:42758          localhost:9877         TIME_WAIT
```

This time we pipe the output of netstat into grep, printing only the lines with our server's well-known port:

- The client's side of the connection (since the local port is 42758) enters the TIME_WAIT state
- The listening server is still waiting for another client connection.

Normal Termination

The following steps are involved in the normal termination of client and server:

When we type our EOF character, fgets returns a null pointer and the function str_cli (Section 5.5) returns.

str_cli returns to the client main function (Section 5.5), which terminates by calling exit.

Part of process termination is the closing of all open descriptors, so the client socket is closed by the kernel. This sends a FIN to the server, to which the server TCP responds with an ACK. This is the first half of the TCP connection termination sequence. At this point, the server socket is in the CLOSE_WAIT state and the client socket is in the FIN_WAIT_2 state (Figure 2.4 and Figure 2.5)

When the server TCP receives the FIN, the server child is blocked in a call to read (Section 3.8), and read then returns 0. This causes the str_echo function to return to the server child main. [Errata] [p128]

The server child terminates by calling exit. (Section 5.2)

All open descriptors in the server child are closed.

The closing of the connected socket by the child causes the final two segments of the TCP connection termination to take place: a FIN from the server to the client, and an ACK from the client.

Finally, the SIGCHLD signal is sent to the parent when the server child terminates.

This occurs in this example, but we do not catch the signal in our code, and the default action of the signal is to be ignored. Thus, the child enters the zombie state. We can verify this with the ps command

Normal Termination


```
linux % ps -t pts/6 -o pid,ppid,tty,stat,args,wchan
PID     PPID    TT      STAT    COMMAND         WCHAN
22038   22036   pts/6   S       -bash           read_chan
17870   22038   pts/6   S       ./tcpserv01     wait_for_connect
19315   17870   pts/6   Z       [tcpserv01 <defu  do_exit
```

The STAT of the child is now Z (for zombie).

We need to clean up our zombie processes and doing this requires dealing with Unix signals. The next section will give an overview of signal handling.